

Datapath

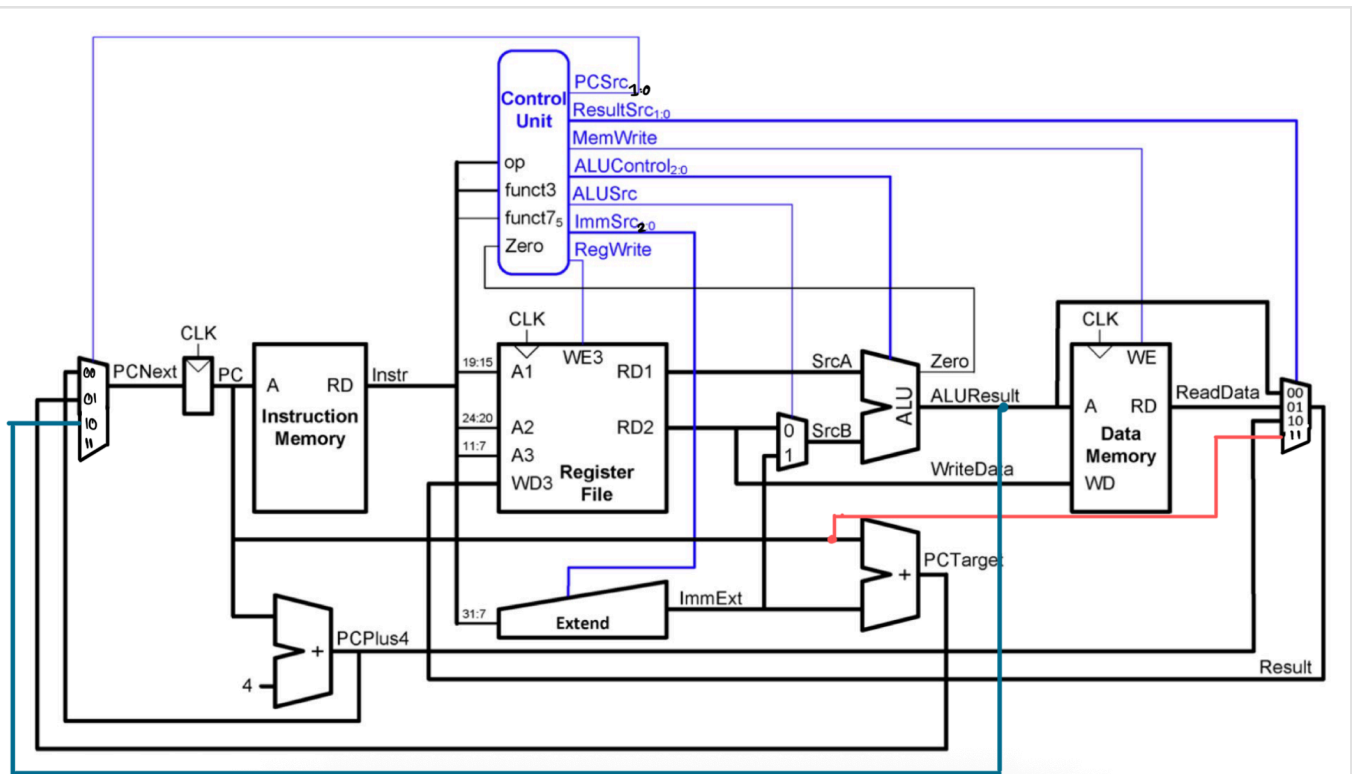


Figure 1: Datapath Diagram

Controller

Instr	Type	opcode	funct3	funct7	ImmSrc	ALUSrc	ResultSrc	PCSrc	MemW	RegW	ALUCtrl
add	R	0110011	000	0000000	—	0	00	00	0	1	ADD
sub	R	0110011	000	0100000	—	0	00	00	0	1	SUB
and	R	0110011	111	0000000	—	0	00	00	0	1	AND
or	R	0110011	110	0000000	—	0	00	00	0	1	OR
slt	R	0110011	010	0000000	—	0	00	00	0	1	SLT
addi	I	0010011	000	—	I(000)	1	00	00	0	1	ADD
xori	I	0010011	100	—	I(000)	1	00	00	0	1	XOR
ori	I	0010011	110	—	I(000)	1	00	00	0	1	OR
slti	I	0010011	010	—	I(000)	1	00	00	0	1	SLT
lw	I	0000011	010	—	I(000)	1	01	00	0	1	ADD
jalr	I	1100111	000	—	I(000)	1	10	10	0	1	ADD
sw	S	0100011	010	—	S(001)	1	00	00	1	0	ADD
beq	B	1100011	000	—	B(010)	0	00	01	0	0	SUB
bne	B	1100011	001	—	B(010)	0	00	01	0	0	SUB
jal	J	1101111	—	—	J(011)	0	10	01	0	1	ADD
lui	U	0110111	—	—	U(100)	1	00	00	0	1	ADD

Table 1: Control Signals

Code Implementation

```
#include <stdint.h>

#define N 20

int32_t arr[N] = {
    12, -5, 33, 7, 0, 19, -12, 44, 8, -1,
    3, 27, 15, 2, -8, 6, 9, -3, 25, 1
};

int main() {
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - 1 - i; j++)
            if (arr[j] > arr[j + 1]) {
                int32_t temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
    }
    return 0;
}
```

C Source

```
addi x1, x0, 0
addi x2, x0, 0
addi x8, x0, 20
addi x4, x0, 19

LOOP1:
    slt x12, x2, x4
    beq x12, x0, DONE
    addi x3, x0, 0
    addi x7, x1, 0
    sub x9, x4, x2

    LOOP2:
        slt x10, x3, x9
        beq x10, x0, END_LOOP2
        lw x5, 0(x7)
        lw x6, 4(x7)
        slt x11, x6, x5
        beq x11, x0, NO_SWAP
        sw x6, 0(x7)
        sw x5, 4(x7)

    NO_SWAP:
        addi x3, x3, 1
        addi x7, x7, 4
        jal x0, LOOP2

    END_LOOP2:
        addi x2, x2, 1
        jal x0, LOOP1

DONE:
```

RISC-V

```
00000093
00000113
01400413
01300213
00412633
04060263
00000193
00008393
402204b3
0091a533
02050463
0003a283
0043a303
005325b3
00058663
0063a023
0053a223
00118193
00438393
fd9ff06f
00110113
fbdf06f
```

Machine Code

Testcase

-5
33
7
0
19
-12
44
8
-1
3
27
15
2
-8
6
7
-3
25
1

Input Data

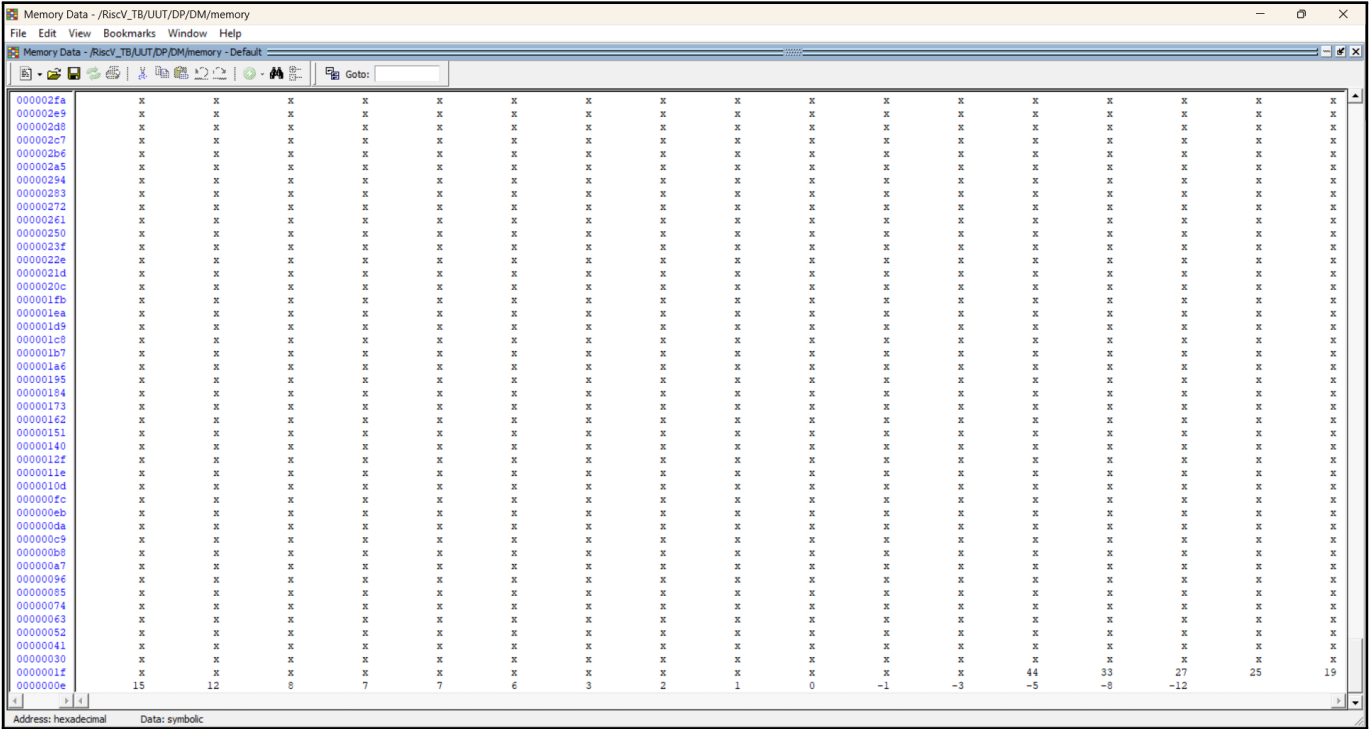


Figure 2: Output Data

RISC-V REFERENCE

James Zhu <jameszhu@berkeley.edu>

RISC-V Instruction Set

Core Instruction Formats

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 ≥ rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 ≥ rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1101111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	

Figure 3: RISC-V Instruction Set